starRoboHarness: A Community-Driven Efficiency Harness Infra for Uniting Robot Policy

starRoboHarness Team

jhhuang117@gmail.com | Code | Project page | HF dataset

Abstract Combining frozen robot policies with reasoning agents requires explicit control over observation timing, action execution, and outcome attribution. We present starRoboHarness, a community-driven execution harness for policy-only, supervised-policy, and direct reasoner control. It binds actions to observations, validates adapter constraints, assigns execution to one controller, and links acknowledged actions to native outcomes. We examine an archived 50-scene RoboDojo study and smaller BEHAVIOR and SimplerEnv diagnostics. The RoboDojo source summaries report 34% and 40% completion for QwenPI and $\pi_{0.5}$ hybrids, versus 18% and 36% for their separately collected 16-step QwenPI and 5-step $\pi_{0.5}$ counterparts. These descriptive rates remain subject to receipt reconciliation and do not isolate a supervision effect. Seven recorded matched supervision pairs show no additional completion, although some interventions increase partial credit. Shorter policy prefixes change task outcomes, while unsuccessful recovery accounts for most recorded correction steps and tokens within accepted $\pi_{0.5}$ hybrid episodes. The harness makes these distinctions inspectable, supporting evaluation of task completion, intervention behavior, and inference cost under explicit execution contracts.

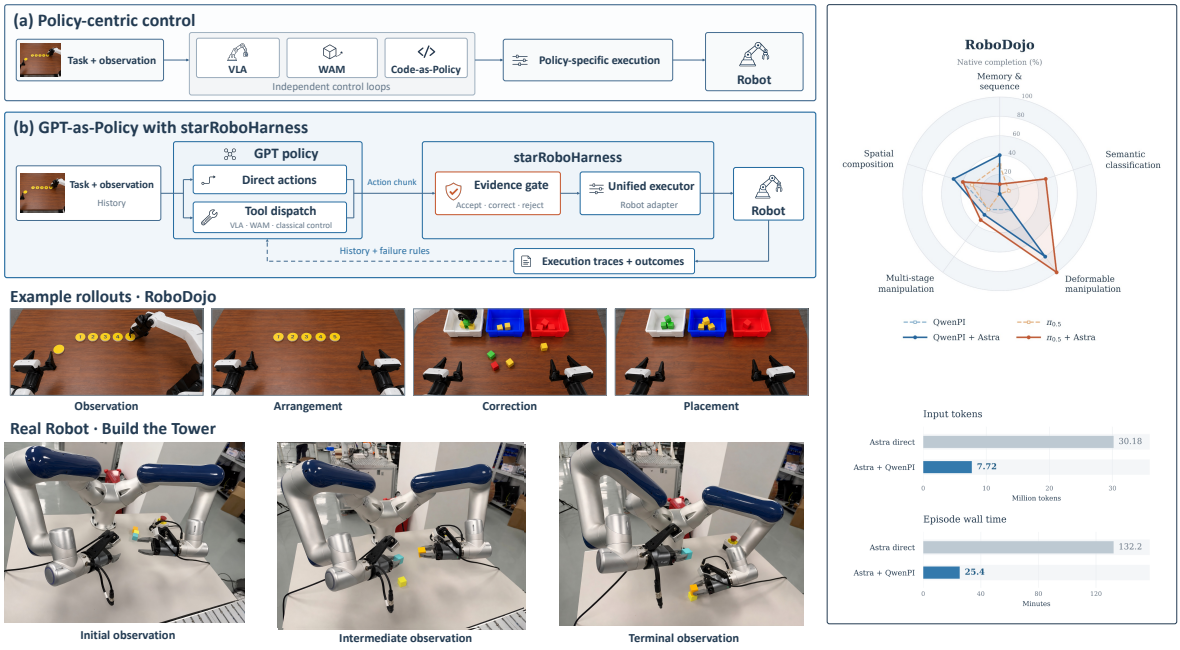


Figure 1 | starRoboHarness connects policy tools and direct reasoner control through an evidence gate and one executor. Simulator rollouts and real-robot photographs illustrate the control setting; the photographs are not a measured physical evaluation. The radar displays four selected series from archived RoboDojo summaries; the full six-configuration comparison is reported separately. The token and wall-time bars come from one RoboDojo build-tower simulator development case per method (Appendix B), not the photographed physical deployment or a formal multi-case study. Input-token snapshots include cached input.

Contents

1 Introduction **3**

2 Related Work **4**

3 Framework **5**
- 3.1 Task setup and notation . . . 5
- 3.2 Decision and execution contract . . . 6
- 3.3 Adapters and community extensions . . . 6
- 3.4 User-facing monitoring and audit . . . 7

4 Experiments **7**
- 4.1 Configurations and saved scenes . . . 7
- 4.2 Outcome definitions and evidence . . . 8

5 Results **9**
- 5.1 Overview of the six configurations . . . 9
- 5.2 Cross-benchmark validation . . . 9
- 5.3 Task group A: five scene-aligned subtasks . . . 10
- 5.4 Task group B: five scene-aligned subtasks . . . 11
- 5.5 Policy choice and replanning cadence . . . 12
- 5.6 $\pi_{0.5}$ with Astra supervision . . . 13
- 5.7 Inference cost and efficiency . . . 13

6 Discussion **15**
- 6.1 Completion does not identify the contribution of correction . . . 15
- 6.2 Cadence and native scoring change the interpretation . . . 16
- 6.3 Recovery cost requires evidence of progress . . . 16
- 6.4 Implications for evaluating a supervisor . . . 16
- 6.5 What the matched audits establish . . . 16
- 6.6 Limits of the evidence . . . 17

7 Conclusion **18**

Contributors **18**

A Ten-task descriptive hybrid index **23**

B Supplementary development cases **23**

C $\pi_{0.5}$ diagnostic tasks and receipt accounting **24**

D Archived $\pi_{0.5}$–Astra scene index **24**
- D.1 First matched pilot diagnostic . . . 25

E Policy complementarity by task **26**

F Intervention and failure trajectories **26**
- F.1 Local recovery in folding and number arrangement . . . 26
- F.2 Semantic redirection in language classification . . . 26
- F.3 Assembly failures and repeated tile recovery . . . 27
- F.4 Packing geometry and rollout variation . . . 28
- F.5 Persistent partial credit and instruction alignment . . . 28

G Reproducibility record **29**

H Data provenance **29**

I Audit-interface examples **31**

J Ongoing benchmark coverage **31**

1. Introduction

Robot control increasingly combines pretrained action models with general-purpose reasoning. Vision-language-action (VLA) policies map images, instructions, and robot state to action sequences [2, 5, 7]. Code as Policies uses a language model to compose executable programs from robot APIs and control primitives [21]. GPT-as-Policy and Agent as Policy allow a general-purpose agent to interact with the environment directly or delegate to a specialist policy [34, 35]. These approaches offer different ways to combine local motor competence, task knowledge, and feedback.

This progression suggests a broader control interface. A reasoner may invoke a VLA for a familiar manipulation skill, use a conventional controller for a constrained motion, or inspect a world-action model before choosing a tool. The same task can require several such operations. A useful system must therefore define how a tool receives the current observation, which proposal may be executed, how long it may retain control, and how its outcome is recorded. Merely sharing a function-call format leaves these questions unresolved.

The missing layer is a robot execution harness. A delayed proposal can refer to an obsolete state. A correction can exceed the action bounds of its robot adapter. Two processes can disagree about whether an action was executed. A reasoner can report completion even though the native evaluator has not accepted the terminal state. These failures arise at the interfaces among policies, reasoning, and execution. They also affect efficiency: frequent replanning can improve feedback while increasing policy requests, and prolonged recovery can consume inference without completing the task.

We introduce starRoboHarness to make these interfaces explicit. Its core contract links an observation, a policy or tool proposal, a validated decision, an execution acknowledgement, and a native outcome. Policy-only execution, supervised policy control, and direct reasoner control share this contract. Benchmark-specific adapters preserve action units, control semantics, and native evaluation. A read-only dashboard connects task outcomes to camera frames and public decision records. The policy checkpoint remains frozen.

The harness is designed for community development. New policy adapters can reuse the contract, while task context, failure examples, and recovery rules can be contributed with their supporting traces. These artifacts can support later integrations without requiring a common policy architecture. We use *community-driven* for this extension mechanism. Accumulating artifacts alone does not establish autonomous self-improvement, and the experiments do not measure a longitudinal learning curve.

Our analysis combines archived summaries for 50 RoboDojo scenes with smaller BEHAVIOR and SimplerEnv diagnostics. The source summaries report higher completion for two supervised policies, together with task-specific regressions. They are descriptive records whose full receipt reconciliation remains open. Matched pilot records clarify what an aggregate rate can conceal: reviews may preserve every action, corrections may increase partial credit without completing the task, and successful hybrid trajectories may contain no corrections. We therefore examine task outcomes and inference use together, keeping archived summaries, matched diagnostics, and ongoing benchmark coverage separate.

Our contributions are:

1. **Community-driven harness infrastructure.** A shared observation, proposal, execution, and receipt contract connects frozen policies and reasoning tools through extensible adapters.
2. **Stability and auditability analysis.** Freshness, bounds, controller ownership, acknowledgements, and native receipts make protocol failures distinguishable from task failure and expose the cost of

intervention.

3. **Cross-benchmark diagnostics.** RoboDojo, BEHAVIOR, and SimplerEnv records test execution paths and distinguish native completion, partial progress, and review overhead. Ongoing RoboTwin, LIBERO-Pro, and RoboCasa365 runs extend coverage under separate protocols.

2. Related Work

Robot policies and action cadence. RT-1 and RT-2 established language-conditioned robot control at scale, while Open X-Embodiment, Octo, and OpenVLA developed cross-embodiment datasets and generalist policy interfaces [1, 2, 3, 4, 5]. Flow and diffusion policies, including π_0, $\pi_{0.5}$, Diffusion Policy, and RDT-1B, use different approaches to continuous action generation [6, 7, 8, 9]. VIMA studies multimodal prompts for manipulation, and BAKU studies efficient multi-task policy learning [10, 11]. Action Chunking Transformers and later real-time chunking work show why the executed prefix and inference cadence belong in the method definition [12, 13]. BridgeData V2 and DROID broaden the training-data base, while LeRobot provides a reusable policy interface [14, 15, 16]. StarVLA supplies the modular policy backbone and the QwenPI_v3 implementation used in our integration, while VLAct studies continued pretraining for vision-language-action policies [17, 18].

Reasoning, supervision, and recovery. SayCan grounds language plans in skill affordances, while Inner Monologue uses environment feedback to revise plans [19, 20]. Code as Policies, PaLM-E, VoxPoser, and ManipLLM connect language or multimodal reasoning to executable robot behavior at different levels of abstraction [21, 22, 23, 24]. Closed-loop reasoning, VLM predictive control, reflective planning, and reasoning-to-decision systems make feedback and replanning explicit [25, 26, 27, 28]. InstructVLA integrates instruction understanding with action generation rather than placing a separate reasoner above a policy [29]. AHA studies vision-language failure detection. RACER and LIBERO-RECOVER address recovery after errors [30, 31, 32]. Runtime monitoring with interactive imitation learning provides another intervention model [33]. GPT-as-Policy compares direct reasoning control with specialist-policy supervision, and Agent as Policy studies a general-purpose agent in direct manipulation control [34, 35]. Harness VLA treats frozen VLAs as reusable manipulation primitives and combines them with memory-guided coding agents [36]. This closely related work motivates shared failure experience and tool composition. starRoboHarness studies the execution contract across policy-only, supervised, and direct control, with explicit links from observations and acknowledgements to native outcomes and inference use. The two systems differ in their control and evaluation scope.

Benchmarks and evaluation validity. RLBench, LIBERO, CALVIN, ManiSkill2, RoboCasa, and RoboMimic provide complementary manipulation suites and datasets [37, 38, 39, 40, 41, 42]. The Colosseum, VLABench, and RoboTwin 2.0 extend evaluation toward generalization, long-horizon reasoning, and bimanual robustness [43, 44, 45]. RoboDojo spans simulation and real-world tasks, and our ten subtasks with five cases each form a selected subset of its evaluation space [46]. SimplerEnv evaluates real-robot policies in visually matched simulation, while BEHAVIOR specifies task-trained policy baselines and a public evaluation protocol [55, 56, 57]. RoboCasa365 extends household simulation to a broader task and scene collection [59]. Benchmark audits and LIBERO-PRO show how task selection and shortcuts affect the meaning of a reported score [47, 48]. UMI-Bench and RB2 emphasize repeatable real-world evaluation and controlled benchmarking [49, 50]. Recommendations for empirical policy evaluation, imperfect simulators, and shared evaluation interfaces motivate our receipt and validity rules [51, 52, 53]. Metamorphic testing provides a related approach to systematic input checks [54]. We retain exact case identity, native receipts, validity rules, and a separate account of excluded attempts. These controls support auditability. The cohorts remain non-contemporaneous and are not a randomized trial.

These studies imply three constraints for a policy–reasoner evaluation. Policy outputs have a temporal granularity, so approving a proposal changes when the system observes and replans. Reasoner interventions consume time and tokens, so quality and cost must be measured on the same cases. A benchmark outcome is meaningful only after the evaluator distinguishes a native timeout from a provider failure. Our case-level record ties these boundaries together and leaves the remaining confounds visible.

3. Framework

Figure 2 presents the architecture. The harness receives the task instruction, current observation, robot state, and action contract. A policy prefix, a supervised decision, or a direct reasoner action passes through the execution gate before one controller calls the benchmark adapter. Each execution returns an acknowledgement; the native evaluator supplies the episode's terminal receipt. The trace links these two levels of evidence. Separately, outcome accounting checks episode identity and receipt provenance before admitting a result to an aggregate. Execution validation and result inclusion serve different purposes.

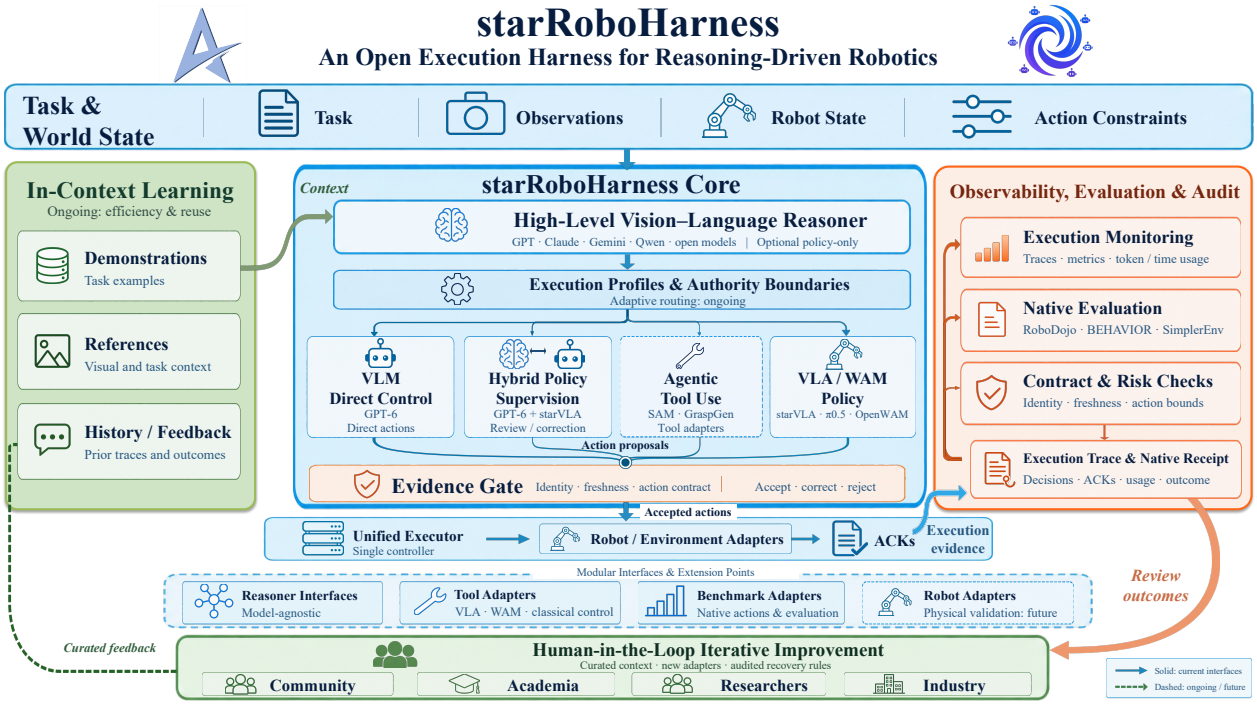


Figure 2 | starRoboHarness execution contract. Policy-only, supervised-policy, and direct reasoner profiles share observation binding and action validation. An accepted decision reaches one environment-facing controller. Execution acknowledgements record what ran, while a native terminal receipt determines the episode outcome. Retained traces support adapter development and audit. Context reuse, adaptive routing, and additional tool profiles are ongoing extensions. The reported diagnostics cover simulation; formal physical validation remains future work.

3.1. Task setup and notation

A task instance is task = $(g, \text{reset}, T, \text{contract})$, where g is the instruction, reset identifies the saved reset, T is the native step budget, and contract specifies the action interface. The contract includes action

dimension, units, coordinate convention, gripper semantics, and adapter bounds. The environment has a hidden state x_t. The controller receives

$$o_t = (\mathrm{id}_t, I_t, q_t, t, T - t), \tag{1}$$

where id_t is the observation identifier, I_t contains camera images, and q_t is the observed robot state. A task scorer may use privileged simulator state, but that state and its reward are excluded from the reasoner's observation interface.

A policy or control tool f_j returns a proposal

$$p_t = (\mathrm{proposal_id}_t, j, \mathrm{id}_t, \mathrm{contract}, U_t), \qquad U_t = (u_{t,0}, \ldots, u_{t,H-1}) \in \mathbb{R}^{H \times d_{\mathrm{contract}}}. \tag{2}$$

Here $\mathrm{proposal_id}_t$ identifies the proposal and j identifies its producer. Prediction horizon H and executed prefix h_t are distinct. The environment advances only by the acknowledged execution of a prefix or a bounded correction. A policy replan without execution does not advance t.

3.2. Decision and execution contract

The public evidence e_t contains previous acknowledgements and retained decision records. A high-level GPT-like reasoner selects a decision $d_t = R(g, o_t, p_t, e_t)$ using the current observation and proposal. In policy-only mode, the configured policy prefix enters the same validation path without a reasoner call. RoboDojo supports delegation, bounded proposal edits, and end-effector correction. The BEHAVIOR review pilot supports accepting the policy output or requesting a fresh plan. In direct control, $p_t = \varnothing$: the decision carries its own bounded action sequence and current observation identity.

Let $V_{\mathrm{contract}}(o_t, p_t, d_t)$ be the adapter's validation predicate. It checks the decision's observation and request identities, finite actions with the expected shape, an admissible execution horizon, and the bounds and semantics defined by contract. When a proposal is present, its identity must also match. Execution follows

$$(o_{t+\Delta t}, a_t) = \mathcal{E}_{\mathrm{contract}}(d_t) \quad \text{only if} \quad V_{\mathrm{contract}}(o_t, p_t, d_t) = 1, \tag{3}$$

where a_t records the executed mode and actual step count $\Delta t \le h_t$. Native termination may interrupt the requested prefix. A failed validation records an error without executing an action. Only the environment-facing controller calls the state-changing interface. A lost or uncertain acknowledgement leaves an unresolved attempt; it must be reconciled before another state-changing request. This contract prevents silent replay rather than assuming exactly-once delivery across a failed transport. Adapter bounds are interface checks, not a physical-safety guarantee.

The episode record is

$$\mathrm{trace}_{\mathrm{task}} = \left\{(o_t, p_t, d_t, a_t, c_t)\right\}_t, \qquad r_{\mathrm{task}} = (z_{\mathrm{task}}, s_{\mathrm{task}}, v_{\mathrm{task}}, n_{\mathrm{task}}), \tag{4}$$

where c_t records available inference usage, and the native receipt r_{task} contains termination status z_{task}, binary success s_{task}, native score v_{task}, and step count n_{task}. The record supports checking what was proposed, accepted, and executed. The receipt determines the benchmark outcome. Neither a verbal success claim nor a high partial score substitutes for native completion.

3.3. Adapters and community extensions

The reference RoboDojo policies receive three RGB views and dual-arm proprioception. QwenPI_v3 produces a 50×14 absolute-joint proposal using its pinned `arx_x5` normalization. The RoboDojo-fine-tuned $\pi_{0.5}$ checkpoint uses its released OpenPI configuration [7]. SimplerEnv uses a different

end-effector interface, while BEHAVIOR uses the normalized 23-dimensional R1Pro contract. The shared record structure therefore does not imply interchangeable actions.

An integration supplies a policy or tool adapter, an environment adapter, and a task configuration. Reusable context can include demonstrations, documented failures, and recovery rules grounded in recorded trajectories. Contributors can extend these components independently of the frozen checkpoint. The current implementation provides episode-local context and trace export. Automatic retrieval, cross-user rule selection, and longitudinal improvement remain extensions requiring separate evaluation.

3.4. User-facing monitoring and audit

The dashboard groups camera playback, task outcomes, execution progress, and public reasoner decisions by task. Selecting a decision seeks to the corresponding retained observation, making a claimed correction inspectable alongside the executed steps. Outcome summaries separate completed native episodes from interrupted attempts. The UI reads experiment artifacts without controlling the robot. It supports analysis and debugging, while the underlying receipts and hashes remain the authoritative evidence.

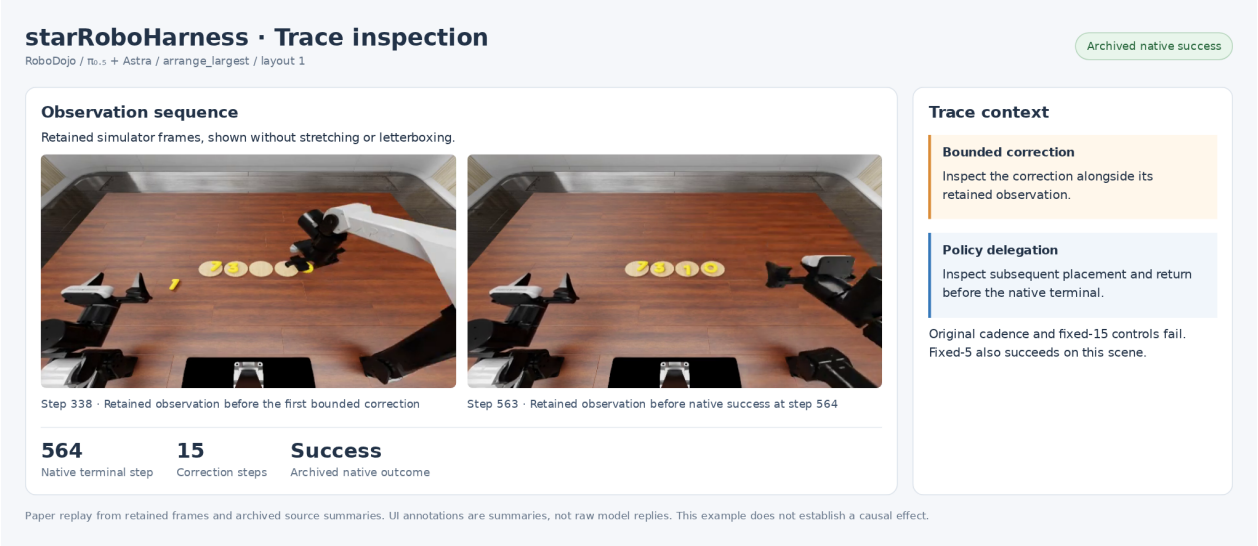


Figure 3 | Read-only paper replay with retained RoboDojo camera frames at their original aspect ratio. A pre-correction observation is shown alongside the later arrangement and archived native success. The annotations summarize the saved trajectory; they are not raw model replies. This example does not isolate the effect of correction.

4. Experiments

4.1. Configurations and saved scenes

The archived RoboDojo summaries cover six configurations: QwenPI_v3, QwenPI_v3 with Astra, pure $\pi_{0.5}$ with 50-, 15-, and 5-action prefixes, and $\pi_{0.5}$ with Astra. The source records describe GPT-6 Astra hybrids with adaptive prefixes of at most 15 steps and corrections bounded to five steps. Pure QwenPI executes 16 steps per prediction. The arms vary both supervision and cadence and were collected separately, so their differences do not identify an Astra-only effect.

Each source configuration lists one outcome for each of 50 saved scenes, ten tasks with five cases

each. Scene alignment checks task, variant, reset, case ID, and scene digest; it does not establish equal policy RNG or collection conditions. The $\pi_{0.5}$ scene manifests align with the QwenPI manifest. Groups A and B retain the QwenPI collection provenance (Tables 3 and 4). The archived exports provide the descriptive counts below. Reconciliation of all historical per-case receipts and runtime identities is still incomplete, and the exports are not evidence that a new 300-episode campaign has finished.

4.2. Outcome definitions and evidence

The intended unit of analysis is a native terminal episode. A verified result requires a terminal receipt, a unique case identity, and agreement with the registered runtime and evaluation protocol. Native timeouts at the step limit remain valid failures with their native partial score. Infrastructure failures, provider failures, pending cases, and uncertain execution remain in a separate attempt ledger. For the archived six-configuration study, the denominators below reproduce the source export's accepted rows; they are not a claim that every historical receipt has been independently recovered. Table checks establish export consistency and scene coverage. Appendix H states the remaining provenance limits.

We report average success rate as the fraction of valid cases that reach native completion and average native score as the mean evaluator score over those cases. Native score captures partial task progress, so it is reported alongside binary completion rather than used as a substitute for it. Scene-level comparisons count successes unique to each configuration and score increases, decreases, and ties. Resource measures include native steps, policy requests, correction steps, elapsed controller time, and reported model tokens. Pure $\pi_{0.5}$ request counts are reconstructed from native steps and executed prefix lengths. Token snapshots include cached input and are not billing records.

The three pure $\pi_{0.5}$ cadences share a checkpoint and native client. The hybrid adds a reasoner and adaptive prefixes. QwenPI and $\pi_{0.5}$ differ in policy interface and serving stack, and hardware, provider sessions, collection period, and recovery history vary. We therefore report scene outcomes and resource use descriptively. Exact scene alignment supports trajectory comparison, but it does not create a randomized supervision or latency study. Appendix J separately records the partial RoboTwin, LIBERO-Pro, and RoboCasa365 coverage available during this revision.

5. Results

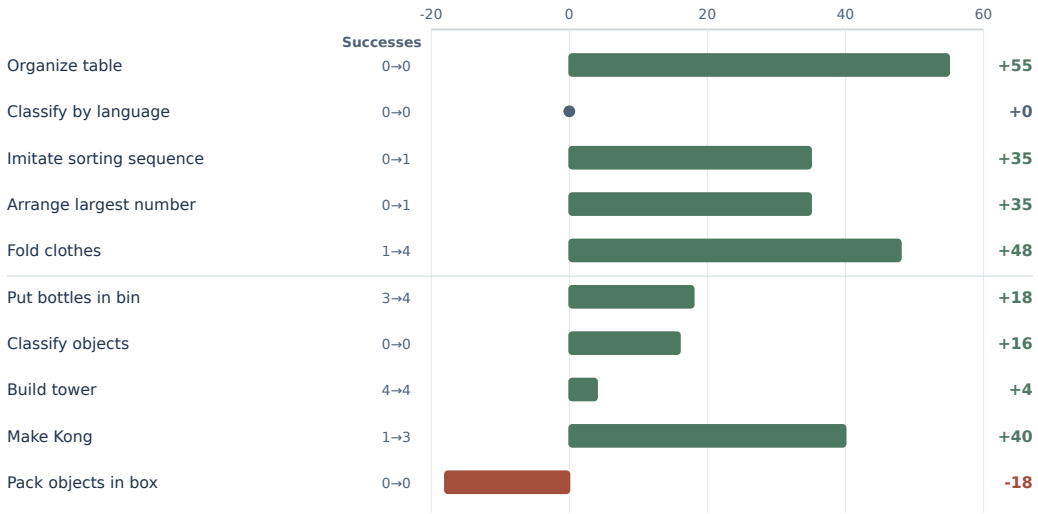


Figure 4 | Task-level change in archived mean native score for scene-aligned RoboDojo groups. Green bars indicate a higher hybrid score, the slate-blue dot marks zero change, and the red bar indicates a lower score. The left labels report policy-to-hybrid completion counts. The figure separates partial-score change from binary completion.

5.1. Overview of the six configurations

Table 1 reproduces 300 outcome entries from the archived source summaries on 50 saved scenes. QwenPI with Astra completes 17 scenes versus nine for pure QwenPI. Pure $\pi_{0.5}$ completes 10, 12, and 18 at the three execution cadences. Its Astra hybrid completes 20. The means and success counts describe the recorded configurations under the comparison conditions in Section 4. They are not isolated effects of either the policy backbone or supervision.

Configuration	Policy prefix	Success	Mean score
QwenPI_v3	16	9/50	23.7
QwenPI_v3 + Astra	Adaptive	17/50	47.0
$\pi_{0.5}$, native cadence	50	10/50	31.8
$\pi_{0.5}$, 15-step cadence	15	12/50	32.0
$\pi_{0.5}$, 5-step cadence	5	18/50	41.4
$\pi_{0.5}$ + Astra	Adaptive	20/50	55.3

Table 1 | Archived source summaries for six configurations on the same 50 saved scenes. Receipt reconciliation is incomplete. Policy prefix is the number of proposed actions executed before replanning. Adaptive hybrid prefixes are at most 15 steps, with corrections bounded to five steps. Success counts native completions, and mean score uses the native 0–100 scale. Each configuration has one separately collected rollout per scene.

5.2. Cross-benchmark validation

The external diagnostics test the execution contract under different action spaces, checkpoints, and native evaluators. Table 2 separates the archived RoboDojo summaries from the smaller receipt-backed

diagnostic records. Their denominators remain separate. A completed diagnostic verifies a tested execution path, not benchmark-wide policy performance.

Benchmark	Tested path	Evidence unit
RoboDojo	Frozen policy with / without Astra	50 saved scenes per configuration
BEHAVIOR	Native / pass-through VLAct	2 public instances
BEHAVIOR	Astra with task-trained policy	2 radio-task diagnostic trials
BEHAVIOR	Direct Astra	1 radio-task diagnostic trial
SimplerEnv	QwenPI with / without Astra	3 seed-0 task pairs
SimplerEnv	Direct Astra	3 seed-0 task trials

Table 2 | Tested execution paths. RoboDojo uses archived source exports; external rows are small native diagnostics. Sample units and protocols remain separate.

The external tracks use different action spaces and checkpoints. BEHAVIOR provides task-trained radio checkpoints and a public evaluation protocol [56, 57]. SimplerEnv-specific NVIDIA GR00T checkpoints are available [58]. The two VLAct native runs exercise the R1Pro adapter and receipt path on public instances 301 and 302. The task-trained radio runs use separate checkpoints. Neither track represents the full BEHAVIOR task set. Conflicting SimplerEnv GR00T source summaries remain unreconciled, so its rate is withheld. The broader policy-only SimplerEnv records report QwenPI 10/15 and RT-1-X 12/15; they are separate from the three seed-0 supervision pairs.

The matched cross-benchmark evidence is operational. On SimplerEnv, the harness pass-through preserves every seed-0 QwenPI action hash, and the Astra pilot preserves the same hashes while completing 22 valid reviews. All three tasks retain the same native outcome, so the review path adds cost without changing control in these cases. On BEHAVIOR, two fresh native/pass-through pairs each validate 3,225 actions and retain failure with Q-score 0. These checks show that the contract and receipt path transfer across evaluators. They do not establish a population-wide absence of benefit. The BEHAVIOR pairs contain no reasoner intervention. The direct pilots use one BEHAVIOR instance and three SimplerEnv tasks.

5.3. Task group A: five scene-aligned subtasks

Task	Policy succ.	Hybrid succ.	Policy score	Hybrid score
Organize table	0/5	0/5	10.0	65.0
Classify by language	0/5	0/5	0.0	0.0
Imitate sorting	0/5	1/5	1.0	36.0
Arrange largest number	0/5	1/5	2.0	37.0
Fold clothes	1/5	4/5	32.0	80.0
Overall (25)	1/25	6/25	9.0	43.6

Table 3 | Task group A. Each task has five aligned saved scenes and separately collected trajectories. Success is shown as completed cases over valid cases. Score is the mean native score on a 0–100 scale.

Hybrid success is 6/25 (24%) versus 1/25 (4%) for the pure policy. Native mean score is 43.6 versus 9.0 (Table 3). Across the aligned scenes there are five hybrid-only wins, no policy-only wins, and 20 ties. Native score improves in 16 pairs, declines in one, and ties in eight. Fold clothes contributes three of the five additional successes. Its success count rises from one to four and its mean score rises from 32 to 80. Language classification is unsuccessful for both methods and has zero mean score under both methods. The aggregate gain combines a task-specific improvement with a task showing

no measurable progress. Different student execution cadences and stochastic visual observations prevent attributing the observed difference solely to corrections.

5.4. Task group B: five scene-aligned subtasks

Task	Policy succ.	Hybrid succ.	Policy score	Hybrid score
Put bottles in bin	3/5	4/5	64.0	82.0
Classify objects	0/5	0/5	6.0	22.0
Build tower	4/5	4/5	82.0	86.0
Make Kong	1/5	3/5	20.0	60.0
Pack objects in box	0/5	0/5	20.0	2.0
Overall (25)	8/25	11/25	38.4	50.4

Table 4 | Task group B. Each task has five aligned saved scenes and separately collected trajectories. Pure-policy counterparts were collected after the fixed hybrid cases. Success is shown as completed cases over valid cases.

Group B gives 11/25 (44%) hybrid and 8/25 (32%) pure-policy successes, with mean native scores 50.4 and 38.4 (Table 4). The 25 aligned scenes contain four hybrid-only wins, one policy-only win, and 20 ties. Score changes are more mixed: seven positive, three negative, and 15 tied pairs. Make Kong accounts for two extra successes, while bottles accounts for one. Build tower has 4/5 successes under both methods. The gain is concentrated in a few tasks.

Packing is the largest regression. Both methods fail all five cases, but mean native partial score falls from 20 to 2. The hybrid cases reach the native 1,300-step limit with scores 0, 0, 0, 0, and 10, so all five are valid scored failures. In one trace, a verified toy-car grasp is followed by repeated five-step transfer corrections, four accepted 15-step policy segments, and a later takeover after an open-gripper hover. The trace includes a verified grasp, so grasping alone does not explain that failure. It motivates analysis of post-grasp transfer and intervention timing, without explaining the other four failures. Classification likewise remains at 0/5 success for both methods despite a higher hybrid partial score.

Figure 5 separates completion from partial progress. Eight tasks have a higher hybrid mean score, one is tied, and packing is lower. In group B classification, score rises from 6 to 22 while success remains 0/5 under both methods. Build tower retains 4/5 successes with a four-point score increase. A larger score can indicate progress toward a goal without a completed task.

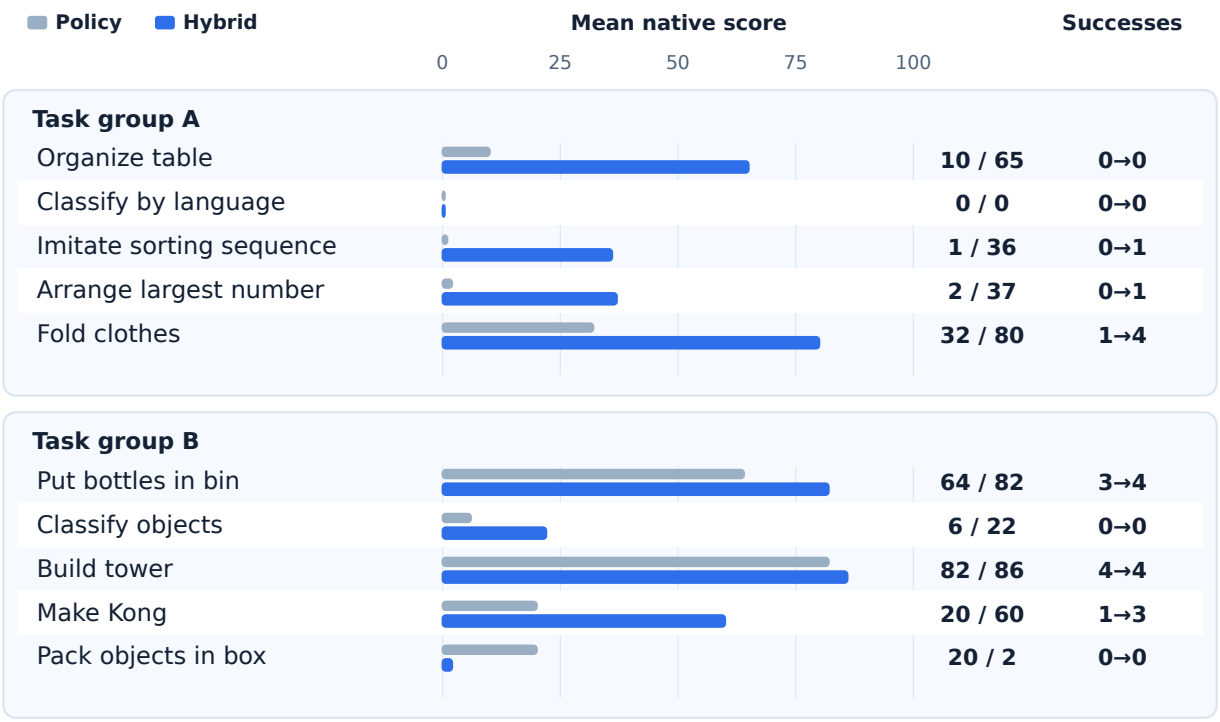


Figure 5 | Archived task-level native score and completion on five aligned saved scenes per task. Gray bars show the pure policy and blue bars the hybrid. The right columns report mean score and policy-to-hybrid completion counts. The plot exposes task-level heterogeneity that an aggregate table would hide. Groups A and B were collected in different periods.

Ten-task descriptive index. Across both groups, the hybrid completes 17/50 cases (34%) and has mean native score 47.0. The corresponding policy cases complete 9/50. Because group B policy cases were collected later than its hybrid cases, these pooled counts describe the observed task set. The group summaries retain their collection periods and are descriptive contrasts.

5.5. Policy choice and replanning cadence

At original cadence, pure $\pi_{0.5}$ completes 10 scenes versus nine for QwenPI. The one-case difference hides six scenes completed only by $\pi_{0.5}$ and five only by QwenPI, with four shared successes. Their union covers 15 scenes. This is retrospective complementarity, not the performance of a runnable selector. The task-level comparison is given in Table 13.

For pure $\pi_{0.5}$, shortening the executed prefix from 50 to 15 to five steps gives 10, 12, and 18 completions, respectively (Table 5). We call these native cadence, 15-step cadence, and 5-step cadence. The older internal labels fixed-15 and fixed-5 refer to these execution protocols, not to different model checkpoints. Both shorter cadences complete three folding scenes and all five bottle-placement scenes. 5-step cadence additionally completes two packing scenes and four build-tower scenes, but neither short cadence completes a Make Kong scene. Language classification and sorting have no completions at any of the three pure-policy cadences. The aggregate trend conceals these task differences.

Task	Orig. succ.	15 succ.	5 succ.	Orig. score	15 score	5 score
Organize table	0/5	1/5	1/5	35	40	35
Classify by language	0/5	0/5	0/5	0	0	2
Imitate sorting	0/5	0/5	0/5	0	0	0
Arrange largest number	0/5	0/5	1/5	3	4	26
Fold clothes	0/5	3/5	3/5	16	64	64
Put bottles in bin	3/5	5/5	5/5	76	100	100
Classify objects	1/5	1/5	2/5	26	36	51
Build tower	3/5	2/5	4/5	68	46	82
Make Kong	3/5	0/5	0/5	60	0	0
Pack objects in box	0/5	0/5	2/5	34	30	54
Source summary	10/50	12/50	18/50	31.8	32.0	41.4

Table 5 | Archived exact-scene pure $\pi_{0.5}$ cadence summaries. Each row contains five saved layouts. Original, 15, and 5 denote executed actions per fresh policy prediction. Scores are mean native RoboDojo scores on a 0–100 scale. The source exports accept 50 rows per method; full receipt reconciliation remains open. Each has one rollout per scene, collected separately. No method uses a reasoner.

5.6. $\pi_{0.5}$ with Astra supervision

The $\pi_{0.5}$ hybrid completes 20/50 scenes with mean score 55.3, compared with 18/50 and 41.4 for 5-step cadence pure policy. Fourteen scenes succeed under both, six only under the hybrid, and four only under 5-step cadence. The hybrid has higher native score on 19 scenes, lower score on seven, and equal score on 24. The six unique hybrid completions comprise two language-classification scenes, two random folding scenes, one object-classification scene, and one Make Kong scene. 5-step cadence alone completes an organize-table scene, a build-tower scene, and two packing scenes.

Table 6 reports task completion and resource use together. The hybrid completes all folding and bottle-placement scenes. It completes two language-classification scenes, which none of the three pure-policy cadences completes. It also fails every packing scene, including the two completed under 5-step cadence. The higher aggregate score and completion count coexist with scene-level regressions. The adaptive hybrid prefixes and separate collection conditions preclude attributing the 13.9-point mean-score difference to Astra alone.

5.7. Inference cost and efficiency

The cost of supervision includes more than the task outcome. The reasoner can add policy replans, correction actions, and model calls. Quality and resource use must therefore be examined on the same cases. Group B retains case-aligned source telemetry. Its wall-time and usage counters remain descriptive because hardware and provider routes were not matched and historical provenance is still being reconciled.

Task	Source cases	Hybrid succ.	Fixed-5 succ.	Hybrid score	Corr. steps	Tokens (M)
Organize table	5/5	0/5	1/5	55.0	1,340	117.52
Classify by language	5/5	2/5	0/5	50.0	819	80.17
Imitate sorting	5/5	0/5	0/5	26.0	879	98.25
Arrange largest number	5/5	1/5	1/5	42.0	890	107.94
Fold clothes	5/5	5/5	3/5	100.0	15	22.36
Put bottles in bin	5/5	5/5	5/5	100.0	5	44.24
Classify objects	5/5	3/5	2/5	66.0	423	100.40
Build tower	5/5	3/5	4/5	62.0	680	92.85
Make Kong	5/5	1/5	0/5	20.0	730	89.89
Pack objects in box	5/5	0/5	2/5	32.0	1,825	156.42
Observed total	50/50	20/50	18/50	55.3	7,606	910.02

Table 6 | Archived $\pi_{0.5}$–Astra summaries for 50 saved scenes. Source cases are rows accepted by the original export, whose receipt reconciliation remains open. The 5-step pure-policy rows were collected separately. Scores use the native 0–100 scale. Corrections combine edited and recovery steps. Reported tokens include cached input and are not billing.

Metric, group B (25 cases)	Pure policy	Hybrid
Mean control-loop wall time (s)	330.5	4,999.8
Mean policy calls / episode	54.4	85.6
Total correction steps	0	4,627
Total input tokens	0	462,535,681
Cached input fraction	n/a	93.95%
Total output tokens	0	1,563,270

Table 7 | Runtime telemetry on group B exact cases. Token counts are the last recorded asynchronous usage snapshots, not billing. Hardware, transport, and collection period differ, so wall-time values are descriptive rather than a controlled slowdown estimate.

Table 7 shows the scale of reasoning used by the group B hybrid. Input-token caching is high, yet 462.5 million reported input tokens and 4,627 correction steps were recorded across 25 cases. The roughly 15-fold difference between mean control-loop times is descriptive because machines and transports differ. Packing shows that additional reasoning and corrections need not improve a native outcome. A controlled comparison must measure quality and cost on matched cases, hardware, and provider routes.

The completed pure $\pi_{0.5}$ cadence panel supplies a within-policy quality and cost check on the same 50 saved scenes. Native client logs imply 884 policy requests over 43,770 steps at original cadence, 2,819 over 42,049 steps at 15-step cadence, and 8,313 over 41,525 steps at 5-step cadence. Completion rises from 10 to 12 to 18 cases, and mean native score is 31.8, 32.0, and 41.4. 5-step cadence uses 2.95 times as many requests as 15-step cadence for six additional observed completions, and 9.40 times as many as original cadence for eight additional completions. Language classification, sorting, and Make Kong together consume 2,665 5-step cadence requests, 32% of that method's total, without a completion. Four other task families account for all six extra 5-step cadence completions over 15-step cadence. Requests are reconstructed from control steps and executed prefix lengths. They do not measure GPU time, energy, or billing. These are single rollouts at each cadence.

Resource use in the 50 $\pi_{0.5}$–Astra cases accepted by the historical export is concentrated in unsuccessful recovery. Twenty-nine failed episodes with corrections account for 7,296 of 7,606 correction steps

and 713.53 of 910.02 million reported tokens. The seven corrected successes use 310 correction steps and 85.64 million tokens. Thus 96% of recorded correction steps and 78% of accepted-case reported tokens fall in corrected failures. Harder scenes can require both more recovery and more inference, so these descriptive groups do not establish that corrections caused failure.

The 910.02 million tokens cover accepted episodes alone. A separate export lists 20 excluded-attempt entries across 13 scene identities. Thirteen entries have usage snapshots, reporting a further 229.18 million tokens; seven have no usage snapshot. This is a lower bound on the reported usage associated with the listed excluded attempts, not an estimate of total campaign cost or billing. Excluded attempts remain outside the task-completion denominator, while their available resource use must remain visible. Reconstructing an all-attempt total requires raw usage records and reconciliation of retry identities and cumulative counters.

The ten scenes with discordant hybrid and 5-step cadence success show this pattern. Two of six hybrid-only successes use no corrections. The other four use 15, 40, 55, and 165 correction steps. In all four 5-step cadence-only successes, the hybrid fails after 290, 380, 450, or 470 correction steps. Correction volume therefore does not track the observed completion advantage on these scenes. The rollouts were collected separately, so task difficulty, action timing, and trajectory variation remain possible explanations.

The concentration also appears by task (Table 6). Organize table, sorting, and packing each have five native-valid hybrid cases and no success. Their fifteen cases use 372.18 million reported tokens and 4,044 correction steps, or 41% of tokens and 53% of correction steps in the complete panel. Mean native partial scores are 55, 26, and 32, respectively. These tasks show that costly partial progress can coexist with zero completion.

On all 50 scenes, the hybrid records 3,952 student-policy predictions and the same number of Astra decisions over 42,539 native steps. 5-step cadence pure policy requires 8,313 reconstructed policy requests over 41,525 native steps on those saved scenes. The hybrid makes 52% fewer policy requests, but adds a reasoner decision for every proposal and reports 910.02 million tokens. This is an operational tradeoff between policy requery frequency and reasoner use. Different trajectories and control horizons prevent interpreting the request ratio as an isolated speed or compute gain.

The supplementary development cases and direct-control case are reported separately in Appendix B. Neither contributes to the six-configuration summaries.

Policy quality also changes the observations and proposals available to the reasoner. A weak proposal can require more recovery, while a strong proposal may only need validation. The harness is designed to reduce dependence on a particular tool through bounded rejection, correction, and tool dispatch; it cannot guarantee that the reasoner compensates for a weak policy. Testing that objective requires matched decision states or controlled policy-quality perturbations.

6. Discussion

6.1. Completion does not identify the contribution of correction

Thirteen of the twenty successful $\pi_{0.5}$ hybrid episodes contain no correction, including four of five folding and four of five bottle-placement successes. They still contain reasoner decisions and report 105.74 million tokens. A completed hybrid trajectory can therefore demonstrate useful delegation without demonstrating successful recovery. Among the six scenes completed only by the hybrid rather than 5-step pure policy, two have no correction and four have 15, 40, 55, or 165 corrected steps.

The folding and language-classification traces localize interventions followed by native success (Ap-

pendix F). They show the executed mechanism, but selection of an intervention depends on the observed trajectory. Without a corresponding control from that decision state, the trace cannot establish that the correction was necessary or sufficient.

6.2. Cadence and native scoring change the interpretation

Execution cadence changes which scenes are completed even with the policy checkpoint fixed. The three pure $\pi_{0.5}$ cadences complete 10, 12, and 18 scenes, but shorter prefixes lose the original cadence's three Make Kong successes. The hybrid also has both gains and regressions relative to 5-step cadence: it completes two language-classification scenes and two random folding scenes absent from the latter, while losing two packing completions. These crossovers make the executed prefix part of the method rather than an incidental serving parameter.

Native score introduces a second distinction. RoboDojo's transition scorers retain reached stages even when later motion disturbs object poses. Organize table and sorting have hybrid mean scores of 55 and 26 despite zero completions. In packing, visible placement is insufficient without the required orientation, containment, box alignment, and open grippers. A controller's verbal assessment cannot replace those checks. The trace must distinguish observed motion, language-specified progress, credited stages, and the final native condition.

6.3. Recovery cost requires evidence of progress

Corrected failures account for 78% of reported tokens and 96% of correction steps in the archived $\pi_{0.5}$ hybrid summary. This concentration does not establish that correction causes failure: difficult scenes can require both more recovery and more inference. It does identify failed recovery as the principal place to inspect resource use. Linking a correction to its precondition, executed action, next observation, and native progress makes repeated visits to an unresolved state distinguishable from recovery that advances the task. The detailed assembly, tile, and packing traces show such unresolved states; reducing their inference budgets remains a hypothesis for controlled evaluation.

6.4. Implications for evaluating a supervisor

The audited panel supports three conclusions about evaluation. A hybrid completion is not evidence that a correction was necessary: thirteen of the twenty successful $\pi_{0.5}$ hybrid episodes contain no corrections. More reasoning is not evidence of effective recovery: fifteen hybrid episodes in organize table, sorting, and packing consume 372.18 million reported tokens without a completion. Finally, exact scene matching does not establish reproducibility of an outcome: 5-step cadence pure policy completes the same packing scene in two of three runs. Each conclusion follows from a counterexample or an audited count, without assuming that the reasoner caused the observed outcome.

These observations define separate questions for a controlled evaluation. Native completion tests whether the full task was achieved. Matched execution windows test whether supervision adds value beyond the policy's replanning cadence. Repeated paired runs quantify outcome variability, while the inference ledger measures the resources spent on both completed episodes and interrupted attempts. Reporting one of these quantities cannot substitute for the others.

6.5. What the matched audits establish

The external matched audits support a limited result. In the three seed-0 SimplerEnv QwenPI pairs, Astra completed 22 valid reviews and selected the student action every time. The action hashes, step

counts, and native outcomes matched the byte-preserving control in all three pairs. The review path therefore added inference latency without changing the executed control or task result in this sample. In the two fresh BEHAVIOR native/pass-through pairs, both arms ran 3,225 steps, received Q-score 0, and failed the task. Those pairs validate gateway preservation and do not contain a reasoner intervention.

Four completed RoboDojo matched pilots provide direct intervention evidence. We multiply the audit exports' fractional scores by 100 to use the same 0–100 scale as the historical tables. On organize_table, both arms failed at 1,000 steps. The control scored 25 and Astra scored 0 after 140 eef intervention steps. On build_tower, both arms failed at 1,050 steps and scored 10. Astra used 780 student steps, 260 eef steps, and 10 edit steps. The original classify_objects pair ended at 1,100 steps with scores 0 and 40 after 235 eef steps and 15 edit steps. A fresh pair on the same frozen scene and protocol again ended at 1,100 steps with scores 0 and 40. This Astra arm used 160 eef and 45 edit steps. The archived audits report that scene identity, reset, cadence, native receipts, and immutable artifact manifests passed for all four pairs. These are within-pair results, not an estimate of the average effect across RoboDojo.

The classify_objects trace shows what the partial-score increase means. Astra intervened when the student trajectory carried a watch toward the vehicle basket, redirected a pen away from the watch basket, and corrected the final vehicle's transport toward the white basket. The controller's final assessment records all three categories in their intended baskets, but the native evaluator still reports a timeout and score 40 on the displayed scale because the full terminal condition was not reached. The audit therefore supports a concrete claim about intermediate recovery in this case. It does not support a claim that the intervention policy reliably completes the task.

The matched SimplerEnv telemetry shows the efficiency tradeoff. The review path used 1,149.5 seconds of recorded reasoner time and 1,188.7 seconds of wall time across the three tasks, compared with 36.5 seconds for the byte-preserving control. This is a descriptive 32.55-fold wall-time ratio for this serving path. It is not a hardware-normalized latency estimate. Zero action edits still required inference cost.

Across the seven policy-supervision pairs, both arms have the same binary outcome. The three SimplerEnv pairs retain the same actions. The four RoboDojo pairs change partial score in different directions and contain no additional completion. The two classification pairs repeat one saved scene, so they provide a repeat check for that scene rather than two independent tasks. The two BEHAVIOR gateway pairs test action preservation without a reasoner. Figure 6 shows the per-pair outcomes without pooling benchmarks.

6.6. Limits of the evidence

The study covers ten selected RoboDojo tasks with five scenes per task. Each configuration has one separately collected rollout per scene, and hardware, serving, provider sessions, and cadence differ. Exact scene files do not guarantee identical trajectories: supplementary 5-step cadence packing repeats complete the same layout in two of three runs, while two frozen stack-bowls runs change from success to failure. The reported counts establish the observed outcomes and cost allocation. Task-level rates are imprecise, and the results describe the recorded embodiments, tasks, and serving conditions.

Matched contrast. For the matched pilots, we report the observed within-pair native outcome difference $Y_i(\text{Astra}) - Y_i(\text{control})$ for the same saved scene, policy checkpoint, execution cadence, reset identity, and serving conditions. The six-configuration panel is descriptive because its arms were collected separately. Episodes without a verified native terminal receipt are excluded from

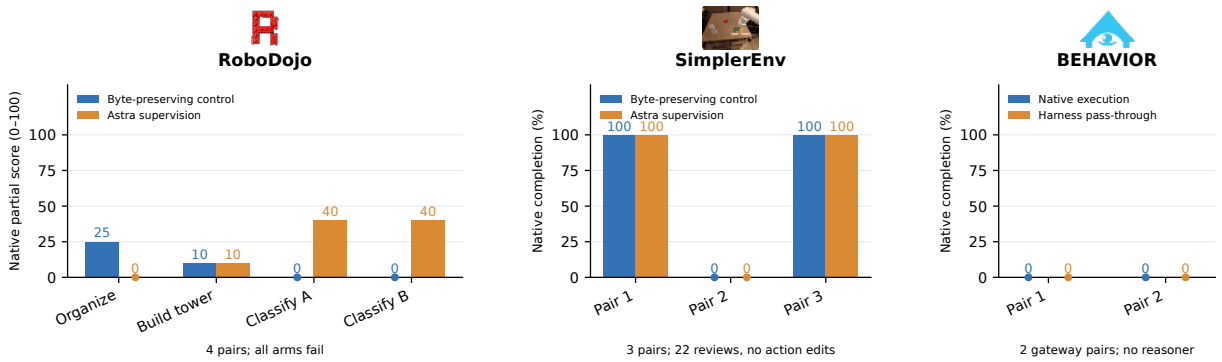


Figure 6 | Matched diagnostic outcomes. RoboDojo bars show per-pair partial scores on a 0–100 scale; all four pairs fail natively. SimplerEnv bars show completion on the three seed-0 tasks; both paths complete two. The BEHAVIOR pass-through pairs both fail and contain no reasoner. Counts and sample scopes remain separate.

outcome denominators. The reported pairs pass scene identity, arm identity, cadence, provenance, and terminal receipt audits.

7. Conclusion

starRoboHarness defines an execution contract for composing frozen robot policies with reasoning and control tools. Observation binding, adapter validation, controller ownership, and native receipts make execution and outcome attribution inspectable. Archived RoboDojo summaries motivate the analysis, while matched pilot records show no additional completion in the seven tested supervision pairs. Partial-score changes, action-preserving reviews, and costly unsuccessful recovery explain why an aggregate hybrid rate cannot establish intervention quality. Ongoing benchmark extensions further require separate treatment of checkpoints, normalization, reset states, and evaluation splits. Repeated controlled runs and complete receipt reconciliation remain necessary before claiming broad improvements in reliability or efficiency. The reusable contribution is the contract and its audit record; community-contributed adapters and recovery rules can then be evaluated against explicit task and resource criteria.

Contributors

Core contributors: Jiehui Huang, Haowen Yan, Jinhui Ye, Yuxin Chen, Zixuan Wang, Senqiao Yang, and Xia Bin.

Advisors: Pengguang Chen, Shu Liu, and Jiaya Jia.

References

[1] Anthony Brohan, Noah Brown, Justice Carbajal, et al. *RT-1: Robotics Transformer for Real-World Control at Scale*. 2023, arXiv:2212.06817, link.

[2] Anthony Brohan, Noah Brown, Justice Carbajal, et al. *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control*. 2023, arXiv:2307.15818, link.

[3] Embodiment Collaboration, Abby O'Neill, Abdul Rehman, et al. *Open X-Embodiment: Robotic Learning Datasets and RT-X Models*. 2025, arXiv:2310.08864, link.

[4] Octo Model Team, Dibya Ghosh, Homer Walke, et al. *Octo: An Open-Source Generalist Robot Policy*. 2024, arXiv:2405.12213, link.

[5] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. *OpenVLA: An Open-Source Vision-Language-Action Model*. 2024, arXiv:2406.09246, link.

[6] Kevin Black, Noah Brown, Danny Driess, et al. *π_0: A Vision-Language-Action Flow Model for General Robot Control*. 2026, arXiv:2410.24164, link.

[7] Physical Intelligence, Kevin Black, Noah Brown, et al. *π_0.5: a Vision-Language-Action Model with Open-World Generalization*. 2025, arXiv:2504.16054, link.

[8] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, Shuran Song. *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion*. 2024, arXiv:2303.04137, link.

[9] Songming Liu, Lingxuan Wu, Bangguo Li, et al. *RDT-1B: a Diffusion Foundation Model for Bimanual Manipulation*. 2025, arXiv:2410.07864, link.

[10] Yunfan Jiang, Agrim Gupta, Zichen Zhang, et al. *VIMA: General Robot Manipulation with Multimodal Prompts*. 2023, arXiv:2210.03094, link.

[11] Siddhant Haldar, Zhuoran Peng, Lerrel Pinto. *BAKU: An Efficient Transformer for Multi-Task Policy Learning*. 2024, arXiv:2406.07539, link.

[12] Tony Z. Zhao, Vikash Kumar, Sergey Levine, Chelsea Finn. *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*. 2023, arXiv:2304.13705, link.

[13] Kevin Black, Manuel Y. Galliker, Sergey Levine. *Real-Time Execution of Action Chunking Flow Policies*. 2025, arXiv:2506.07339, link.

[14] Homer Walke, Kevin Black, Abraham Lee, et al. *BridgeData V2: A Dataset for Robot Learning at Scale*. 2024, arXiv:2308.12952, link.

[15] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. *DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset*. 2025, arXiv:2403.12945, link.

[16] Remi Cadene, Simon Aliberts, Francesco Capuano, et al. *LeRobot: An Open-Source Library for End-to-End Robot Learning*. 2026, arXiv:2602.22818, link.

[17] StarVLA Community. *StarVLA: A Lego-like Codebase for Vision-Language-Action Model Developing*. 2026, arXiv:2604.05014, link.

[18] Senqiao Yang, Chengyao Wang, Yuxin Chen, et al. *Beyond Data Scaling: Representation-Centric Continued Pre-training for Vision-Language-Action Models*. 2026, arXiv:2608.27550, link.

[19] Michael Ahn, Anthony Brohan, Noah Brown, et al. *Do As I Can, Not As I Say: Grounding Language in Robotic Affordances*. 2022, arXiv:2204.01691, link.

[20] Wenlong Huang, Fei Xia, Ted Xiao, et al. *Inner Monologue: Embodied Reasoning through Planning with Language Models*. 2022, arXiv:2207.05608, link.

[21] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, Andy Zeng. *Code as Policies: Language Model Programs for Embodied Control*. 2023, arXiv:2209.07753, link.

[22] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, et al. *PaLM-E: An Embodied Multimodal Language Model*. 2023, arXiv:2303.03378, link.

[23] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, Li Fei-Fei. *VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models*. 2023, arXiv:2307.05973, link.

[24] Xiaoqi Li, Mingxu Zhang, Yiran Geng, et al. *ManipLLM: Embodied Multimodal Large Language Model for Object-Centric Robotic Manipulation*. 2023, arXiv:2312.16217, link.

[25] Michal Nazarczuk, Jan Kristof Behrens, Karla Stepanova, Matej Hoffmann, Krystian Mikolajczyk. *Closed Loop Interactive Embodied Reasoning for Robot Manipulation*. 2025, arXiv:2404.15194, link.

[26] Wentao Zhao, Jiaming Chen, Ziyu Meng, Donghui Mao, Ran Song, Wei Zhang. *VLMPC: Vision-Language Model Predictive Control for Robotic Manipulation*. 2024, arXiv:2407.09829, link.

[27] Yunhai Feng, Jiaming Han, Zhuoran Yang, Xiangyu Yue, Sergey Levine, Jianlan Luo. *Reflective Planning: Vision-Language Models for Multi-Stage Long-Horizon Robotic Manipulation*. 2025, arXiv:2502.16707, link.

[28] Yifu Yuan, Haiqin Cui, Yibin Chen, et al. *From Seeing to Doing: Bridging Reasoning and Decision for Robotic Manipulation*. 2026, arXiv:2505.08548, link.

[29] Shuai Yang, Hao Li, Bin Wang, et al. *InstructVLA: Vision-Language-Action Instruction Tuning from Understanding to Manipulation*. 2026, arXiv:2507.17520, link.

[30] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, et al. *AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation*. 2024, arXiv:2410.00371, link.

[31] Yinpei Dai, Jayjun Lee, Nima Fazeli, Joyce Chai. *RACER: Rich Language-Guided Failure Recovery Policies for Imitation Learning*. 2024, arXiv:2409.14674, link.

[32] Lin Liu, Zhicheng Bao, Lu Zhang, et al. *LIBERO-RECOVER: Beyond Task Success Towards Failure Recovery in Robotic Manipulation Models*. 2026, arXiv:2609.05178, link.

[33] Huihan Liu, Shivin Dass, Roberto Martín-Martín, Yuke Zhu. *Model-Based Runtime Monitoring with Interactive Imitation Learning*. 2023, arXiv:2310.17552, link.

[34] GPT-as-Policy contributors. *GPT as Policy*. 2026, link.

[35] Mengzhao Jia, Yang Lin, Xixin Zhang, Zhihan Zhang, Xiaobai Liu, Meng Jiang. *Agent as Policy for Robotic Manipulation*. 2026, arXiv:2609.12541, link.

[36] Yixian Zhang, Huanming Zhang, Feng Gao, et al. *Harness VLA: Steering Frozen VLAs into Reliable Manipulation Primitives via Memory-Guided Agents*. 2026, arXiv:2607.08448, link.

[37] Stephen James, Zicong Ma, David Rovick Arrojo, Andrew J. Davison. *RLBench: The Robot Learning Benchmark & Learning Environment*. 2019, arXiv:1909.12271, link.

[38] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, Peter Stone. *LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning*. 2023, arXiv:2306.03310, link.

[39] Oier Mees, Lukas Hermann, Erick Rosete-Beas, Wolfram Burgard. *CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks*. 2022, arXiv:2112.03227, link.

[40] Jiayuan Gu, Fanbo Xiang, Xuanlin Li, et al. *ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills*. 2023, arXiv:2302.04659, link.

[41] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, Yuke Zhu. *RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots*. 2024, arXiv:2406.02523, link.

[42] Ajay Mandlekar, Danfei Xu, Josiah Wong, et al. *What Matters in Learning from Offline Human Demonstrations for Robot Manipulation*. 2021, arXiv:2108.03298, link.

[43] Wilbert Pumacay, Ishika Singh, Jiafei Duan, Ranjay Krishna, Jesse Thomason, Dieter Fox. *THE COLOSSEUM: A Benchmark for Evaluating Generalization for Robotic Manipulation*. 2024, arXiv:2402.08191, link.

[44] Shiduo Zhang, Zhe Xu, Peiju Liu, et al. *VLABench: A Large-Scale Benchmark for Language-Conditioned Robotics Manipulation with Long-Horizon Reasoning Tasks*. 2024, arXiv:2412.18194, link.

[45] Tianxing Chen, Zanxin Chen, Baijun Chen, et al. *RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation*. 2025, arXiv:2506.18088, link.

[46] Tianxing Chen, Yue Chen, Zixuan Li, et al. *RoboDojo: A Unified Sim-and-Real Benchmark for Comprehensive Evaluation of Generalist Robot Manipulation Policies*. 2026, arXiv:2607.04434, link.

[47] Tianchong Jiang, Xiangshan Tan, Samuel Wheeler, Luzhe Sun, Tewodros W. Ayalew, Matthew Walter. *What Are We Actually Benchmarking in Robot Manipulation?*. 2026, arXiv:2606.04233, link.

[48] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, Lichao Sun. *LIBERO-PRO: Towards Robust and Fair Evaluation of Vision-Language-Action Models Beyond Memorization*. 2026, arXiv:2510.03827, link.

[49] Shi Jin, Yuntian Wang, Yuhui Duan, et al. *UMI-Bench 1.0: An Open and Reproducible Real-World Benchmark for Tabletop Robotic Manipulation with UMI Data*. 2026, arXiv:2606.10382, link.

[50] Sudeep Dasari, Jianren Wang, Joyce Hong, et al. *RB2: Robotic Manipulation Benchmarking with a Twist*. 2022, arXiv:2203.08098, link.

[51] Hadas Kress-Gazit, Kunimatsu Hashimoto, Naveen Kuppuswamy, Paarth Shah, Phoebe Horgan, Gordon Richardson, Siyuan Feng, Benjamin Burchfiel. *Robot Learning as an Empirical Science: Best Practices for Policy Evaluation*. 2024, arXiv:2409.09491, link.

[52] Apurva Badithela, David Snyder, Lihan Zha, Joseph Mikhail, Matthew O'Kelly, Anushri Dixit, Anirudha Majumdar. *Reliable and Scalable Robot Policy Evaluation with Imperfect Simulators*. 2025, arXiv:2510.04354, link.

[53] XPolicyLab Community, Tianxing Chen, Yue Chen, et al. *XPolicyLab: A Unified Standard and Open Ecosystem for Robot Policy Evaluation and Deployment*. 2026, arXiv:2608.09892, link.

[54] Pablo Valle, Sergio Segura, Shaukat Ali, Aitor Arrieta. *Metamorphic Testing of Vision-Language Action-Enabled Robots*. 2026, arXiv:2602.22579, link.

[55] Xuanlin Li, Kyle Hsu, Jiayuan Gu, et al. *Evaluating Real-World Robot Manipulation Policies in Simulation*. 2024, arXiv:2405.05941, link.

[56] BEHAVIOR Challenge contributors. *2026 BEHAVIOR Challenge Baselines*. 2026, link.

[57] BEHAVIOR Challenge contributors. *2026 BEHAVIOR Challenge Evaluation and Rules*. 2026, link.

[58] NVIDIA. *Isaac-GR00T: An Open Foundation Model for Generalist Humanoid Robot Policies*. 2025, link.

[59] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, Yuke Zhu. *RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots*. International Conference on Learning Representations, 2026, arXiv:2603.04356, link.

[60] RoboCasa core team. *Overview of Benchmarking: Training and Evaluation Protocols*. Official RoboCasa v1.0.1 documentation, 2026, accessed 10 October 2026, link.

A. Ten-task descriptive hybrid index

Group	Task	Hybrid succ.	Hybrid score
A	Organize table	0/5	65.0
A	Classify by language	0/5	0.0
A	Imitate sorting	1/5	36.0
A	Arrange largest number	1/5	37.0
A	Fold clothes	4/5	80.0
B	Put bottles in bin	4/5	82.0
B	Classify objects	0/5	22.0
B	Build tower	4/5	86.0
B	Make Kong	3/5	60.0
B	Pack objects in box	0/5	2.0
Total	50 cases	17/50	47.0

Table 8 | Archived summary of 50 selected hybrid cases by task group. Success is shown as completed cases over valid cases. This descriptive index is not a separate experiment.

B. Supplementary development cases

Dev task	Native result	Score	Steps	Policy calls	Corrections
Classify by language	Timeout	10	1,100	146	481
Arrange largest number	Success	100	487	34	0
Make Kong	Success	100	381	27	0
Total / mean	2/3 success	70	1,968	207	481

Table 9 | Supplementary development validation: one standard `g0/10` layout per task. Score in the final row is the three-case mean. Other final-row quantities are totals.

The development evaluation completes two of three standard `g0/10` tasks (Table 9). All 481 correction steps occurred in the failed language-classification episode. Both successes required none. The receipts report 30.91 million input tokens, including 29.83 million cached input tokens, and 72.6 thousand output tokens. No simultaneous pure-policy comparator was run, so this small development sample cannot establish a correction benefit.

Supplementary `build_tower` case	Native result	Score	Steps	Policy calls
QwenPI_v3, 16-step	Timeout	10	1,050	66
QwenPI_v3, 15-step cadence ablation	Timeout	0	1,050	70
Astra + QwenPI_v3	Success	100	761	52
Astra Direct	Timeout	0	1,050	0

Table 10 | Supplementary development evidence for one `build_tower` layout. Astra Direct issued 210 direct decisions. Its zero policy-call count reflects the absence of a student policy. No row belongs to the frozen panels.

In the supplementary `build_tower` case, Astra plus QwenPI_v3 succeeded with zero correction steps, whereas pure QwenPI_v3 and Astra Direct timed out (Table 10). This single layout checks

native termination across execution paths but cannot establish a population effect. The rounded input-token snapshots were 7.72 million for the hybrid and 30.18 million for Astra Direct, including 7.41 million and 29.90 million cached input tokens, respectively. Episode wall times were 25.4 and 132.2 minutes. These values underlie the bars in Figure 1; they reflect this simulator development comparison, not final provider billing or a physical-robot evaluation.

C. $\pi_{0.5}$ diagnostic tasks and receipt accounting

Native task	Exploratory	Frozen run 1	Frozen run 2
Stack bowls	2/3 (71.7)	3/3 (100.0)	2/3 (71.7)
Stack blocks	0/3 (15.0)	0/3 (10.0)	0/3 (10.0)
Stack blocks by language	0/3 (6.7)	0/3 (0.0)	0/3 (0.0)
Pour liquid into cup	1/3 (33.3)	1/3 (33.3)	1/3 (33.3)
Pour by language	0/3 (0.0)	0/3 (0.0)	0/3 (0.0)

Table 11 | Supplementary pure $\pi_{0.5}$ diagnostics. Cells give native successes out of three and mean score. The frozen runs share 15 verified scenes and one checkpoint.

The two frozen runs have four and three successes on 15 exact scenes. Fourteen pairs retain their score and outcome. Stack-bowls layout 2 changes from success at 100 to failure at 15. This flip limits claims from one rollout per layout. Block stacking and language-conditioned pouring fail in both runs, whereas ordinary pouring succeeds on one layout. These distinct tasks do not isolate language understanding. None enters the 50-case panel.

The accepted $\pi_{0.5}$ panel uses RoboDojo `Pi_05 59999` in one simulator. All 50 scene hashes match the QwenPI panel. Unstable batches substituted two Make Kong scenes and one excluded sorting scene. We reran the selected layouts. Substitutions and five setup failures enter no denominator.

The full pure-policy cadence comparison appears in Table 5.

D. Archived $\pi_{0.5}$–Astra scene index

Table 12 preserves the 50 hybrid entries in the source export. Native timeouts are recorded as failures. The listing does not establish that all historical receipts and runtime identities have been recovered.

Table 12 | Archived outcomes and resource use for 50 scene-aligned cases. Coverage refers to source entries, not independently reconciled receipts. Corrections combine edited and recovery steps; tokens include cached input. Unlabeled rows use standard layout 0; Rand. denotes the random variant.

Task	Pure $\pi_{0.5}$	$\pi_{0.5}$ + Astra	Steps	Time (s)	Corr.	Tokens (M)
Organize table	Fail / 75	Fail / 50	1,000	4,320	330	28.47
Organize table (layout 1)	Fail / 50	Fail / 25	1,000	5,336	210	35.81
Organize table (layout 2)	Fail / 0	Fail / 50	1,000	2,635	470	17.61
Organize table (layout 3)	Fail / 25	Fail / 75	1,000	1,833	125	11.76
Organize table (layout 4)	Fail / 25	Fail / 75	1,000	3,003	205	23.87
Classify by language	Fail / 0	Success / 100	711	1,800	55	15.44
Classify by language (layout 1)	Fail / 0	Fail / 10	1,100	2,197	210	14.17
Classify by language (layout 2)	Fail / 0	Fail / 40	1,100	2,122	105	23.64
Classify by language (layout 3)	Fail / 0	Fail / 0	1,100	2,834	284	14.84

Continued on next page

Task	Pure $\pi_{0.5}$	$\pi_{0.5}$ + Astra	Steps	Time (s)	Corr.	Tokens (M)
Classify by language (layout 4)	Fail / 0	Success / 100	942	1,659	165	12.09
Imitate sorting	Fail / 0	Fail / 50	1,564	7,878	385	44.78
Imitate sorting (layout 1)	Fail / 0	Fail / 0	442	632	0	5.12
Imitate sorting (layout 2)	Fail / 0	Fail / 50	1,600	2,515	113	18.14
Imitate sorting (layout 3)	Fail / 0	Fail / 30	1,600	3,290	370	22.77
Imitate sorting (layout 5)	Fail / 0	Fail / 0	536	940	11	7.44
Arrange largest number	Fail / 15	Fail / 15	1,050	3,038	200	28.68
Arrange largest number (layout 1)	Fail / 0	Success / 100	564	1,183	15	11.52
Arrange largest number (layout 2)	Fail / 0	Fail / 25	1,050	3,144	145	23.26
Arrange largest number (rand. 0)	Fail / 0	Fail / 30	1,050	4,430	220	26.41
Arrange largest number (rand. 1)	Fail / 0	Fail / 40	1,050	2,956	310	18.07
Fold clothes	Fail / 20	Success / 100	300	650	0	3.95
Fold clothes (layout 1)	Fail / 20	Success / 100	303	585	0	4.84
Fold clothes (layout 2)	Fail / 20	Success / 100	296	652	0	3.75
Fold clothes (rand. 0)	Fail / 20	Success / 100	304	813	15	5.41
Fold clothes (rand. 1)	Fail / 0	Success / 100	302	782	0	4.41
Put bottles in bin	Success / 100	Success / 100	477	642	0	9.38
Put bottles in bin (layout 1)	Success / 100	Success / 100	323	1,317	0	7.76
Put bottles in bin (layout 2)	Fail / 40	Success / 100	440	875	0	7.51
Put bottles in bin (layout 3)	Success / 100	Success / 100	419	780	0	6.84
Put bottles in bin (layout 4)	Fail / 40	Success / 100	700	1,514	5	12.74
Classify objects	Fail / 15	Success / 100	786	1,122	0	13.01
Classify objects (layout 1)	Success / 100	Success / 100	779	1,073	0	14.15
Classify objects (layout 2)	Fail / 0	Fail / 15	1,100	2,542	215	27.66
Classify objects (layout 3)	Fail / 0	Fail / 15	1,100	3,231	193	26.87
Classify objects (layout 4)	Fail / 15	Success / 100	890	1,567	15	18.71
Build tower	Fail / 30	Success / 100	723	931	0	6.09
Build tower (layout 1)	Success / 100	Fail / 0	1,050	4,116	380	32.70
Build tower (layout 2)	Success / 100	Fail / 10	1,050	4,192	300	30.01
Build tower (layout 3)	Success / 100	Success / 100	717	1,219	0	11.48
Build tower (layout 4)	Fail / 10	Success / 100	714	1,271	0	12.57
Make Kong	Success / 100	Fail / 0	600	2,884	120	24.71
Make Kong (layout 1)	Fail / 0	Success / 100	407	1,159	40	9.72
Make Kong (layout 2)	Fail / 0	Fail / 0	600	1,887	120	13.51
Make Kong (layout 3)	Success / 100	Fail / 0	600	3,014	290	23.98
Make Kong (layout 4)	Success / 100	Fail / 0	600	2,380	160	17.96
Pack objects in box	Fail / 10	Fail / 25	1,300	6,082	450	39.37
Pack objects in box (layout 1)	Fail / 50	Fail / 10	1,300	2,635	380	21.62
Pack objects in box (layout 2)	Fail / 10	Fail / 25	1,300	3,136	265	18.50
Pack objects in box (rand. 0)	Fail / 50	Fail / 50	1,300	3,796	290	36.57
Pack objects in box (rand. 1)	Fail / 50	Fail / 50	1,300	4,734	440	40.37
50 source cases					7,606	910.02

D.1. First matched pilot diagnostic

A matched pilot compares 5-step cadence control with Astra supervision on organize-table layout 1. Both arms use the same checkpoint, reset identity, hardware, and 1,000-step horizon. Initial robot state, end-effector pose, instruction, and remaining horizon are bitwise equal. Both episodes time out after 200 decision windows. On the report's 0–100 scale, control scores 25 and Astra scores 0, a difference of −25 points. Astra executes 140 correction steps, while control executes none. Terminal receipts, cadence, scene identity, and artifact hashes pass the audits. Camera arrays and the first policy proposals differ slightly across fresh processes, so matching the reset does not reproduce an identical policy trajectory. This pair documents a supervised timeout despite substantial correction. It does not estimate the average effect of supervision or the effect of an individual correction. The pilot is reported as a supplementary matched case and does not enter the six-configuration panel.

E. Policy complementarity by task

Task	QwenPI		QwenPI + Astra		$\pi_{0.5}$	
	Success	Score	Success	Score	Success	Score
Organize table	0/5	10.0	0/5	65.0	0/5	35.0
Classify by language	0/5	0.0	0/5	0.0	0/5	0.0
Imitate sorting	0/5	1.0	1/5	36.0	0/5	0.0
Arrange largest number	0/5	2.0	1/5	37.0	0/5	3.0
Fold clothes	1/5	32.0	4/5	80.0	0/5	16.0
Put bottles in bin	3/5	64.0	4/5	82.0	3/5	76.0
Classify objects	0/5	6.0	0/5	22.0	1/5	26.0
Build tower	4/5	82.0	4/5	86.0	3/5	68.0
Make Kong	1/5	20.0	3/5	60.0	3/5	60.0
Pack objects in box	0/5	20.0	0/5	2.0	0/5	34.0
All tasks	9/50	23.7	17/50	47.0	10/50	31.8

Table 13 | Three observed methods on the same 50 saved RoboDojo scene layouts. Success is the number of completed cases, and Score is the mean native score on a 0–100 scale. The $\pi_{0.5}$ rows come from a separate pure-policy campaign.

F. Intervention and failure trajectories

The cases below link recorded decisions to native observations and outcomes. They illustrate the executed mechanisms and unresolved failure states. They do not supply the counterfactual trajectory required to establish that a correction was necessary or sufficient.

F.1. Local recovery in folding and number arrangement

On random folding layout 0, head and wrist observations after the first 15 student steps place the left gripper over a yellow book instead of the white sleeve. Three five-step end-effector corrections move the hand inward. The next policy proposal approaches the cuff, and delegated actions fold and release the garment before native success at step 304. The episode has 289 student steps and 15 corrected steps. Three 5-step cadence pure-policy runs, including the main-panel run, fail with score zero on the saved layout. On random layout 1, the hybrid instead succeeds without corrections at step 302, while all three pure cadences fail. The two hybrid folding successes therefore involve different control paths.

On number-arrangement layout 1, three five-step corrections align and lower digit 0 over the rightmost pad and move a held digit 1 toward the third pad. Remaining policy actions produce native success at step 564. The episode reports 11.52 million tokens, including 11.28 million cached-input tokens. Figure 7 shows the arrangement before correction and near termination. Original and 15-step cadence pure policy each fail at score zero, while 5-step cadence also succeeds. That successful pure-policy run demonstrates that this saved scene admits a completed trajectory without Astra.

F.2. Semantic redirection in language classification

Language-classification layout 0 requires watches in the middle basket and wooden toys in the right basket. Eleven five-step corrections redirect a held watch and interrupt proposed transfers of a wooden toy toward the middle basket. The supervisor stages that toy on clear tabletop, then delegates 13



(a) Step 338: two digits placed (b) Step 563: 7310 on the pads

Figure 7 | Native first-camera frames from number-arrangement layout 1. The first precedes the reasoner's first bounded correction. The second shows all four digits on their target pads shortly before the native success at step 564. The frames describe one trajectory and do not isolate the effect of supervision.

policy prefixes that complete placements and arm return. Figure 8 shows the visible sequence. All three pure cadences fail on the saved scene, while the hybrid receives native success and score 100.

On layout 4, the supervisor uses 145 corrected steps by step 260 to stage and redirect a pepper after proposals toward an incorrect basket. Later delegated actions complete the placements, with native success at step 942 after 165 corrected steps in total and 12.09 million reported tokens. The hybrid completes two of five language-classification scenes, has mean score 50, and uses 819 corrected steps and 80.17 million tokens. These successful redirections coexist with three native failures, so the individual trace does not establish transfer across the task.



(a) Step 470: toy above middle basket (b) Step 700: objects in assigned baskets

Figure 8 | High-camera observations from the native-successful $\pi_{0.5}$–Astra language-classification layout. The task requires watches in the middle basket and wooden toys in the right basket. Between these views, the supervisor stages a wooden toy for regrasp before delegating the final placements. The sequence is trajectory evidence, not a controlled causal comparison.

F.3. Assembly failures and repeated tile recovery

Build-tower layout 1 reaches the 1,050-step limit with score zero after 380 corrected steps and 32.70 million reported tokens. Its 76 five-step corrections revisit displaced supports and board placement. Late observations show a board standing on its edge across the supports. The subsequent corrections do not produce a completed stack. Original and 5-step cadence pure $\pi_{0.5}$ succeed on this saved scene. The hybrid's three build-tower completions contain no corrections, while its two failed scenes contain 680 corrected steps in total.

Make Kong has one hybrid completion in five scenes, versus three at original pure-policy cadence and none at either shorter cadence. In the successful hybrid layout 1, eight corrections target a displaced upright matching tile. Later observations record all three matching tiles face-up before success at step 407. This episode uses 40 recovery steps and 9.72 million tokens. Layout 0 instead uses 120 recovery steps on a dropped matching tile and fails at the 600-step limit, reporting 24.71 million tokens.

Layout 2 isolates a repeated recovery sequence. Three matching South tiles are face-up by step 262, but an unrelated five-dot tile is knocked face-up during subsequent placement. From step 435 onward, 105 of the episode's 120 corrected steps attempt to grasp, rotate, and restore that tile. It remains face-up in the final frame, and native score is zero. The trace identifies the unresolved disturbance and where the remaining horizon was spent.

F.4. Packing geometry and rollout variation

Packing layout 1 times out at 1,300 steps with score 10, after 920 student steps and 380 corrected steps. It makes 140 reasoner decisions and reports 21.62 million tokens. At step 908, the controller assesses that all four objects are at the box but leaves their orientations unverified. It then uses 310 of the remaining 392 steps on 62 corrections directed at a displaced toothbrush. Native credit never advances beyond one-object placement. Original and 15-step cadence pure policy also fail on this scene, with scores 50 and 25. The geometric checks, rather than the controller's verbal assessment, determine benchmark progress.

On random packing layout 0, the hybrid fails with score 50 after 290 recovery steps and 36.57 million tokens. Three 5-step cadence pure-policy runs on the saved scene yield two successes and one timeout at score 50. The repeated control exposes variability even with the scene and cadence fixed. On standard layout 0, 5-step cadence succeeds while the hybrid fails at score 25 after 450 corrected steps. Across five packing scenes, the hybrid completes none and uses 1,825 corrections and 156.42 million tokens.

F.5. Persistent partial credit and instruction alignment

Sorting layouts 0 and 2 reach score 50 at the fourth ordered-prefix transition but fail the task. The hybrid uses 385 corrected steps and 44.78 million tokens on the first, versus 113 steps and 18.14 million tokens on the second. Layout 1 contains no correction: all 442 control steps are delegated, yet its 30 reasoner decisions report 5.12 million tokens and its native score is zero. The task has no hybrid completion, mean score 26, and 98.25 million reported tokens. Equivalent partial scores can therefore conceal different trajectories and costs.

Organize-table layout 2 uses 470 corrected steps and reports 17.61 million tokens before timeout at score 50. Of those corrections, 105 steps attempt to grasp and pull the drawer handle. The instruction asks for drawer opening, while the native completion check combines four object-placement conditions and arm return without a separate drawer-opening condition. 5-step cadence pure policy succeeds on this saved scene. Layout 4 reaches score 75 but fails after 205 corrected steps and 23.87 million tokens. That persistent stage records three of four item checks with open grippers. Completion separately requires all four checks and arm return.

Native scoring also distinguishes motion from credited placement. Language classification requires an entire category in the correct basket, no objects from other categories there, and open grippers. Sorting requires a correctly ordered prefix in the target basket with later objects outside it. Packing requires prescribed object orientation and containment, an upright aligned box, and open grippers. A zero score need not imply no object moved, and a nonzero stage need not certify the final arrangement.

G. Reproducibility record

The harness stores one append-only record for every decision window. The following pseudocode specifies the shared control boundary. Validation errors and rejected decisions are recorded without execution. Adapter implementations supply the native acknowledgement and terminal-receipt fields.

Listing 1 | Adapter-neutral execution skeleton.

```
observation = adapter.observe()  # fresh observation identity
proposal = None
if profile.has_policy_tool:
    proposal = producer.propose(observation)
    proposal.validate(contract)
decision = profile.decide(observation, proposal, trace)
if decision.mode == "reject":
    return trace.record_rejection(observation, proposal, decision)
contract.validate_decision(observation, proposal, decision)
ack = controller.execute_once(decision)  # sole state-changing owner
trace.append(observation, proposal, decision, ack)
if not ack.execution_resolved:
    return attempt_ledger.record_unresolved(ack)  # do not replay
if ack.native_terminal:
    return adapter.read_native_terminal_receipt(ack)
```

A policy review prompt contains only the task instruction, the current camera and proprioceptive observation, the previous acknowledged action, and public trace evidence. The reviewer must return a structured decision with a mode, an execution horizon, a bounded correction when allowed, and a short evidence field. Hidden simulator state, future frames, and private chain-of-thought are not part of the interface.

Listing 2 | Sanitized review contract used by the policy-supervision adapters.

```
Input: task, fresh_observation, student_proposal, acknowledged_trace
Return: {mode: accept|correct|reject,
        execute_steps: integer,
        correction: bounded_action_or_null,
        evidence: short_observable_justification}
Rules: use the fresh observation and keep actions in adapter bounds.
       Return control after the requested prefix. Never claim success
       without a native terminal receipt.
```

Configuration files record the checkpoint identifier, action-space name, prefix policy, correction bound, native horizon, provider identity, and source digest. Paths, credentials, account identifiers, and transport endpoints are intentionally omitted. The public repository contains the adapters, validators, audit scripts, and table-generation checks needed to reproduce the reported summaries.

H. Data provenance

The archived RoboDojo exports cover ten selected tasks with five saved scenes per task. Their scene manifests align, and Group A and Group B retain separate collection provenance. These exports support reproduction of the printed descriptive summaries. Independent reconciliation of historical per-case receipts, policy seeds, code snapshots, and runtime identities remains incomplete. The six-configuration numbers therefore remain source-summary observations, not newly completed formal experiments.

Matched pilot audit files are retained separately from those exports. The four RoboDojo pairs cover

three saved scenes, including one repeated classification scene. The SimplerEnv supervision records contain three seed-0 pairs. BEHAVIOR native/pass-through pairs test preservation without a reasoner. Their limited scope is part of each reported result. Conflicting GR00T summaries remain excluded pending checkpoint and receipt reconciliation.

For a formal result, the audit must link the unique case, reset, policy checkpoint, executed code and dependencies, action contract, seed protocol, raw log, and native terminal receipt. Native timeouts retain their evaluator score. Provider failures, transport aborts, and unresolved execution stay outside outcome denominators. A replacement run receives a new trial identity and cannot be counted alongside the superseded attempt. Different benchmark splits and source variants retain separate ledgers.

I. Audit-interface examples

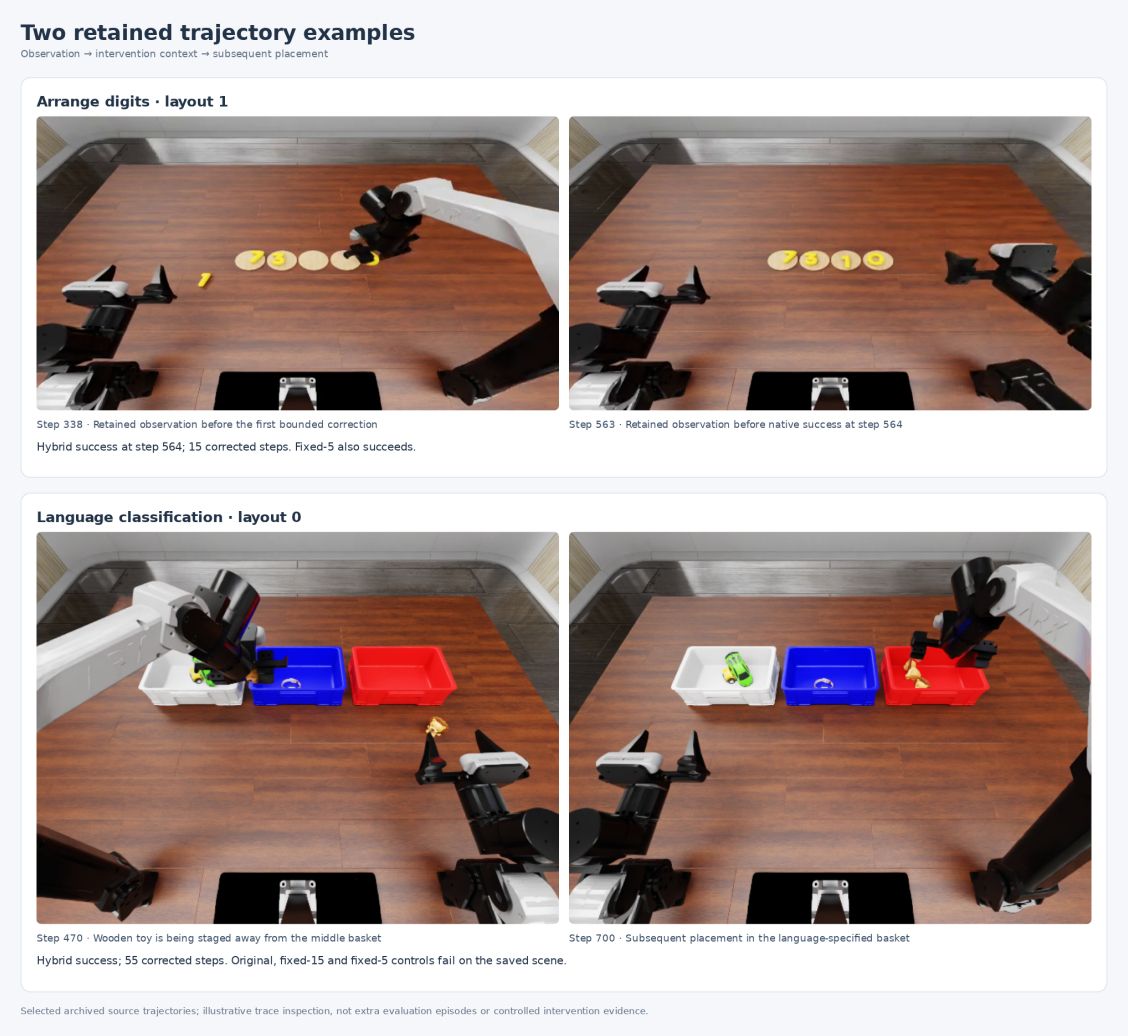


Figure 9 | Additional paper replay views. Retained number-arrangement and language-classification frames show observations before correction and after subsequent placement. These selected examples illustrate trace inspection; they do not add episodes or establish benchmark-wide superiority. Native outcomes and source scope remain explicit in each view.

J. Ongoing benchmark coverage

Table 14 records the audited coverage available on 10 October 2026. These runs extend environment and policy integration to RoboTwin 2.0, LIBERO-Pro, and RoboCasa365. They do not contain matched reasoner comparisons and do not enter the preceding RoboDojo or supervision denominators. Selection of completed cohorts can bias an interim aggregate, so the table reports coverage and observed counts without presenting a full-benchmark ranking.

Official RoboCasa365 evaluation specifies task set, split, task-specific horizons, and 50 sampled scenarios per task [59, 60]. Our integration's registry snapshot provides evaluation metadata for 317 of 365 tasks; 48 lack split or horizon metadata. Exploratory runs with a 7,200-action budget are excluded from benchmark scores. The newer multi-reset pretrain campaign fixes model seed 42 and retains RLinf post-success execution, differing from upstream success-stop evaluation. Corrected

Track	Audited observations	Scope and remaining work
RoboTwin restored contract	953/1,700 successes	17/50 complete 100-episode task cohorts. The earlier normalization-mismatched runs are excluded from this track.
LIBERO clean baseline	454/800 successes	Partial coverage of 2,000 available baseline states. Model RNG restarts per suite and phase.
LIBERO-Pro perturbations	357/1,760 successes	Partial coverage of 14,880 perturbation states. No full task archive is yet covered; six empty archives remain setup gaps.
RoboCasa365 target split	6/50 successes	One seed-7 scene per target task. This is narrow task coverage, not the prescribed multi-episode benchmark evaluation.
RoboCasa365 pretrain extension	30/266 successes	Separate non-target tasks; one reset-repaired task is recorded separately at 0/1. The source variant is retained.

Table 14 | Interim policy-integration coverage, not complete benchmark results. Numerators count observed native successes; denominators count audited episodes or reset states in the stated track. Counts from different splits, checkpoints, and runtime variants are not pooled.

RoboTwin normalization likewise requires a separate execution track, without implying a reasoner improvement.